

Project Implementation Overview

The proposed system will be developed as a centralized, AI-driven campus portal that integrates technical event management, placement management, resume analysis, personalized recommendations, automated notifications, and analytics dashboards. The portal will use a synthetic dataset to simulate learner profiles, resume content, technical events, and placement opportunities, thereby enabling recommendation and filtering features without relying on sensitive real-world records.

Step 1: Define the Project Scope

The first step is to clearly define the scope of the portal. The system should support three user roles: learner, placement coordinator, and administrator. The learner interface will provide profile management, resume upload, technical event recommendations, placement notifications, and career guidance, while the coordinator and administrator interfaces will handle event creation, placement posting, user management, and analytics monitoring.

This role-based structure ensures clear separation of responsibilities and improves system security, usability, and maintainability.

Step 2: Prepare the Synthetic Dataset

Since real institutional records are often unavailable or confidential, a synthetic dataset should be created for development and testing. The dataset should include learner profiles, resume text, skills, certifications, project details, technical event records, placement opportunity records, application history, notifications, and analytics logs.

The synthetic dataset can be prepared in Excel, CSV, or database format. For each learner profile, include fields such as learner ID, name, department, year, CGPA, skills, interests, certifications, project summaries, resume content, and participation history. For technical events, include event ID, title, domain, required skills, eligibility criteria, date, organizer, and description. For placement opportunities, include company name, role, package, required skills, CGPA threshold, eligible branches, application deadline, and selection stages.

The dataset should be realistic, varied, and privacy-safe. Names, identifiers, contact details, and academic information must be fictional and should not correspond to real individuals.

Step 3: Develop the Authentication Module

The next step is to implement user authentication and role-based access control. This module should allow secure registration, login, and session management for learners, coordinators, and administrators. After login, each role should be redirected to a separate dashboard with only the functions permitted for that role.

This structure ensures data privacy, reduces unauthorized access, and makes the portal easier to manage.

Step 4: Build the Learner Profile Module

The learner profile module should allow each user to maintain academic and career-related details. The profile should include skills, interests, projects, certifications, academic performance, resume upload, and participation history. This information will later be used by the recommendation engine and the career guidance module.

A resume upload option should also be included so the system can analyze the document and extract relevant keywords, skills, and project descriptions.

Step 5: Implement Resume Analysis

The resume analysis module should process the uploaded resume and identify the learner's technical strengths and improvement areas. The system can extract skills, tools, programming languages, certifications, project descriptions, and domain interests from the uploaded document.

The extracted information can then be compared against available technical events and placement opportunities to determine relevance and eligibility. This module also supports career guidance by highlighting missing skills, weak project descriptions, and important keywords that should be added to the resume.

Step 6: Create the Technical Event Module

The technical event module should allow coordinators to publish events such as hackathons, coding competitions, workshops, technical seminars, project exhibitions, and interview-preparation sessions. Each event should contain a title, topic, domain, date, description, eligibility rules, registration deadline, and required skills.

Learners should be able to browse events, register for eligible sessions, and receive event recommendations based on resume content and profile data. This makes the portal personalized and useful for technical career development.

Step 7: Create the Placement Module

The placement module should manage company postings, job roles, eligibility criteria, application deadlines, and recruitment stages. Coordinators should be able to upload placement opportunities, and learners should be able to view only the drives that match their academic and skill profile.

This module should also track application status, shortlisting, and offer outcomes. Such information is important for analytics and future recommendation improvement.

Step 8: Develop the Recommendation Engine

The recommendation engine is the core AI component of the system. It should use the resume profile, skills, interests, project history, and academic data to recommend relevant technical events, training sessions, internships, and placement opportunities.

A content-based recommendation approach is suitable for the initial version. In this approach, each event or placement opportunity is tagged with skill keywords and domain labels, and the learner profile is matched against these tags to calculate relevance. For example, a profile with Python, data science, and machine learning keywords should receive recommendations for AI workshops, ML hackathons, and related placement drives.

Step 9: Add Career Guidance Features

The portal should include a career guidance section that provides feedback on resume quality and career readiness. This section can suggest missing technical skills, weak project descriptions, useful certifications, and better keyword usage for applicant tracking and recruiter visibility.

The guidance module should also recommend which technical events or training sessions would help strengthen the profile. This converts the portal from a simple recommendation system into a career-support platform.

Step 10: Implement Notifications

The system should send automated notifications for eligible technical events, matching placement drives, deadline reminders, and resume improvement suggestions. These notifications can be delivered through in-app alerts, email, or both.

The notification module ensures that relevant opportunities reach the intended user in a timely manner and improves participation and engagement.

Step 11: Build Analytics Dashboards

The analytics module should provide dashboards for coordinators and administrators. These dashboards should visualize event participation, placement applications, eligibility counts, skill distribution, and overall engagement trends. The portal can also display placement success rate, event category popularity, and resume improvement statistics.

Charts and summary cards should be used to present the data in a clean and readable format. This helps in decision-making and also strengthens the demonstration of the project.

Step 12: Store Data in a Database

All portal data should be stored in a structured database such as MySQL or MongoDB. The database should contain tables or collections for users, profiles, resumes, technical events, placement opportunities, applications, notifications, analytics logs, and recommendation results.

A well-structured database will make the portal stable, scalable, and easier to maintain. It will also simplify query operations for analytics and recommendation generation.

Step 13: Define the Folder Structure

The implementation should be organized into the following modules:

- `app.py` for the main application.
- `auth/` for login and role management.
- `profiles/` for learner data and resume upload.
- `events/` for technical event management.
- `placements/` for placement opportunity management.
- `recommendation/` for AI matching logic.
- `career_guidance/` for resume improvement suggestions.
- `notifications/` for alerts and messaging.
- `analytics/` for charts and reports.
- `db/` for models and queries.
- `templates/` for HTML files.
- `static/` for CSS, JavaScript, and images.
- `uploads/` for resume files.
- `logs/` for activity and usage records.
- `models/` for any saved ML artifacts.

This modular structure improves maintainability and makes the project easier to present clearly.

Step 14: Choose the Technology Stack

A practical technology stack for this project is:

- Frontend: HTML, CSS, Bootstrap, or React.
- Backend: Flask, Django, or Node.js.
- Database: MySQL or MongoDB.
- AI/ML: Python with scikit-learn.
- Resume parsing: `pdfplumber`, `PyMuPDF`, or `python-docx`.
- Visualization: Plotly or Chart.js.
- Notifications: SMTP email or in-app alert system.

This stack is suitable for a final-year project because it combines full-stack development with AI-based recommendation and analytics capabilities.

Step 15: Implement in Sequence

The project should be implemented in the following order:

1. Authentication and role management.
2. Learner profile creation.
3. Resume upload and parsing.
4. Technical event module.

5. Placement module.
6. Recommendation engine.
7. Career guidance module.
8. Notification system.
9. Analytics dashboard.
10. Testing and refinement.

This order is logical because the AI modules depend on structured profile and resume data.

Step 16: Evaluate the System

Once the portal is complete, it should be tested using the synthetic dataset. Evaluation should focus on recommendation relevance, eligibility filtering accuracy, notification effectiveness, and dashboard correctness. It is also useful to review whether the resume guidance module correctly identifies missing skills and weak areas.

A small set of test cases should verify that the right event or placement opportunity is shown for the right profile type.

Step 17: Prepare the Final Demonstration

The final demonstration should include:

- user login and role selection,
- resume upload,
- profile analysis,
- recommended technical events,
- recommended placement opportunities,
- career guidance output,
- notification display,
- analytics dashboard.

This gives a complete end-to-end flow and clearly shows the AI value of the system.

Final Academic Wording

The project can be formally described as follows:

The proposed AI-driven campus portal integrates technical event management, placement opportunity management, resume analysis, personalized recommendations, automated notifications, and analytics dashboards into a single platform. A synthetic dataset will be used to simulate learner profiles, resume records, technical events, and placement opportunities, ensuring privacy, reproducibility, and practical system evaluation.

Free Dataset Usage

Free resume datasets can also be used for the resume analysis component. Public datasets available from open sources can support resume parsing, skill extraction, resume categorization, and career guidance logic. However, such datasets usually cover resumes only, not technical events or placement drives.

Therefore, the most effective approach is to combine a free resume dataset with a synthetic dataset for events, placements, eligibility rules, notifications, and user interactions. This gives realistic resume data while allowing full control over the portal logic.

Final Implementation Notes

The resume or CV should be treated as the main profile source, because it contains skills, projects, certifications, and interests that drive recommendations. Technical events should be tagged properly using domains such as Python, AI/ML, cloud, cybersecurity, coding contest, or hackathon so that matching becomes possible. Placement opportunities should include eligibility rules such as branch, CGPA, year, and required skills.

The first version should use a simple recommendation logic before attempting any advanced model. The simplest and most effective version is to keep the system resume-aware, technical-event focused, and placement-oriented, with clear eligibility filtering and practical career guidance.

Database, API, and UI Design

Database Design

The database should be normalized and divided into separate tables or collections for users, profiles, resumes, events, placements, applications, notifications, recommendations, and analytics logs.

Core tables include:

- users
- profiles
- resumes
- events
- placements
- applications
- notifications
- recommendations
- analytics_logs

Example Field Design

users

- user_id
- full_name

- email
- password_hash
- role
- created_at

profiles

- profile_id
- user_id
- department
- year
- cgpa
- skills
- interests
- certifications
- projects
- participation_history

resumes

- resume_id
- user_id
- file_name
- file_path
- extracted_text
- parsed_skills
- parsed_projects
- upload_date

events

- event_id
- title
- category
- domain
- required_skills
- eligibility_year
- eligibility_branch
- date
- deadline
- organizer
- description

placements

- placement_id
- company_name
- role
- package
- required_skills

- min_cgpa
- eligible_branches
- application_deadline
- selection_stages
- description

applications

- application_id
- user_id
- opportunity_type
- opportunity_id
- status
- applied_on

API Endpoints

The backend should follow REST-style endpoints with clear actions for create, read, update, and delete operations.

Authentication

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout

Profile and Resume

- GET /api/profile/:id
- PUT /api/profile/:id
- POST /api/resume/upload
- POST /api/resume/analyze

Event Management

- GET /api/events
- GET /api/events/:id
- POST /api/events
- PUT /api/events/:id
- DELETE /api/events/:id

Placement Management

- GET /api/placements
- GET /api/placements/:id
- POST /api/placements
- PUT /api/placements/:id
- DELETE /api/placements/:id

Recommendation and Guidance

- GET /api/recommendations/:userId
- GET /api/career-guidance/:userId
- GET /api/skill-gap/:userId

Notifications and Analytics

- GET /api/notifications/:userId
- POST /api/notifications/send
- GET /api/analytics/summary
- GET /api/analytics/events
- GET /api/analytics/placements

UI Page List

Public pages

- Login page.
- Registration page.
- Forgot password page.

Learner pages

- Dashboard.
- Profile page.
- Resume upload page.
- Resume analysis page.
- Recommended technical events page.
- Placement opportunities page.
- Career guidance page.
- Notifications page.
- Application status page.

Coordinator pages

- Dashboard.
- Create event page.
- Create placement page.
- View applications page.
- Notification management page.
- Analytics page.

Administrator pages

- User management page.
- Role management page.
- Global analytics page.
- Dataset management page.
- System settings page.

Sample Dummy Records

Example Learner Record

- user_id: U101
- name: Arjun
- department: CSE
- year: 3
- cgpa: 8.4
- skills: Python, SQL, HTML, CSS
- interests: AI, web development
- certifications: NPTEL Python
- projects: Smart Attendance System
- participation_history: 3 events

Example Event Record

- event_id: E201
- title: AI Hackathon
- category: Technical
- domain: Machine Learning
- required_skills: Python, ML
- eligibility_year: 3, 4
- eligibility_branch: CSE, IT
- deadline: 2026-08-10

Example Placement Record

- placement_id: P301
- company_name: TechNova
- role: Software Engineer
- package: 6 LPA
- required_skills: Java, DSA, SQL
- min_cgpa: 7.5
- eligible_branches: CSE, IT
- application_deadline: 2026-08-15

Recommendation Scoring Rules

The recommendation engine should begin with a simple rule-based or content-based scoring model.

Basic scoring logic

- Matching skill: 3 points.
- Matching domain interest: 2 points.
- Matching project keyword: 2 points.
- Matching branch or year eligibility: 3 points.
- Matching certification: 1 point.
- Matching CGPA condition: 3 points.

Ranking rule

- Score above 8: highly recommended.
- Score 5 to 8: recommended.
- Score below 5: low priority.

This model is easy to explain and suitable for the first version.

Resume Parsing Rules

Resume parsing should extract structured information from unstructured resume text.

What to extract

- Skills.
- Programming languages.
- Tools and frameworks.
- Projects.
- Certifications.
- Education details.
- Keywords related to role interest.

How to extract

- Convert PDF or DOCX to text.
- Clean the text by removing extra spaces and symbols.
- Match against keyword lists for skills and technologies.
- Identify section headers such as Education, Skills, Projects, and Certifications.
- Store parsed output separately in the database.

Output format

- extracted_text
- skill_list
- project_list
- certification_list
- keyword_score
- missing_skill_flags

Implementation Workflow

The actual coding order should be:

1. Create database tables.
2. Create authentication APIs.
3. Create profile and resume upload APIs.
4. Add resume parsing logic.
5. Add event and placement CRUD APIs.
6. Add filtering logic.
7. Add recommendation scoring.
8. Add career guidance suggestions.
9. Add notifications.

10. Add analytics summaries.
11. Build frontend pages.
12. Test with dummy data.

This order prevents dependency issues and keeps the project modular.

Test Cases

The system should be tested with both matching and non-matching profiles.

Recommended test cases

- A learner with Python and ML skills should receive AI event suggestions.
- A learner with weak resume keywords should receive improvement suggestions.
- A learner with CGPA below threshold should be filtered from certain placements.
- A learner with matching branch and skills should see the highest-ranked placement.
- A learner with no matching skills should see general training events instead.

These cases verify that filtering, ranking, and guidance are working properly.

What to Document in the Report

The report should clearly include:

- database schema,
- API list,
- UI page list,
- dummy dataset format,
- scoring logic,
- parsing logic,
- test cases,
- deployment plan.

This shows that the project is not only conceptual but also implementable.

Final Note

The project will be implemented using a modular web architecture in which the database stores learner profiles, resumes, technical events, placement opportunities, applications, recommendations, and analytics logs. RESTful APIs will manage authentication, profile updates, resume parsing, event management, placement management, recommendation generation, notifications, and analytics. A synthetic or partially public dataset will be used to test the system, while recommendation scores will be calculated through a content-based matching mechanism using skills, projects, interests, eligibility rules, and academic criteria.